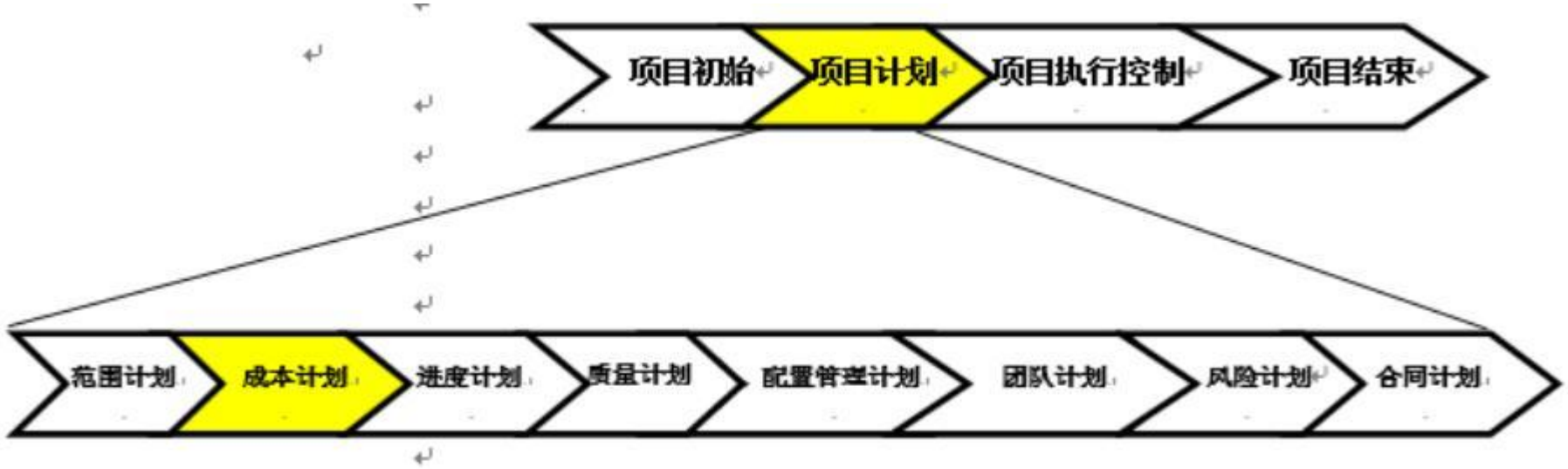
# IT项目管理

史建锋

2025年11月24日

# 成 本 计 划


项目初始
项目计划
项目执行控制
项目结束
范围计划
成本计划
进度计划
质量计划
配置管理计划
团队计划
风险计划
合同计划

# 第 6 章

# IT项目成本计划与控制

# 本 章 要 点

一 估算过程概念

二 传统估算方法

三 敏捷估算方法

四 AI开发范式下的成本估算

五 成本预算

六 案例分析

# 成本估算非常重要

**成本度量——**

**软件产业老大难问题**

**1. 软件市场——乱**

- 前期低价中标，占地盘
- 后期漫天要价，绑架客户

**2. 预算审批——愁**

- 缺少科学客观的评估依据
- 烫手山芋，六拍法

**3. 商务谈判——难**

- 双方无法界定软件成本
- 缺乏合理的议价依据

**4. 项目结算——扯**

- 无法就变更范围达成一致
- 变更内容的计价难题



# IT项目成本：完成IT项目相应付出的代价



- 待开发的项目需要的资金。
- 人劳动消耗所需要的代价是IT项目尤其是软件产品的主要成本

# 估 算 过 程

规模估算 → 工作量估算 → 成本估算

## 软件规模单位

- LOC(Lines of Code)：源代码长度的测量
- FP(Function Point)：用系统的功能数量来测量
- UCP(Use Case Point)：用例点数量测量

## 软件工作量单位

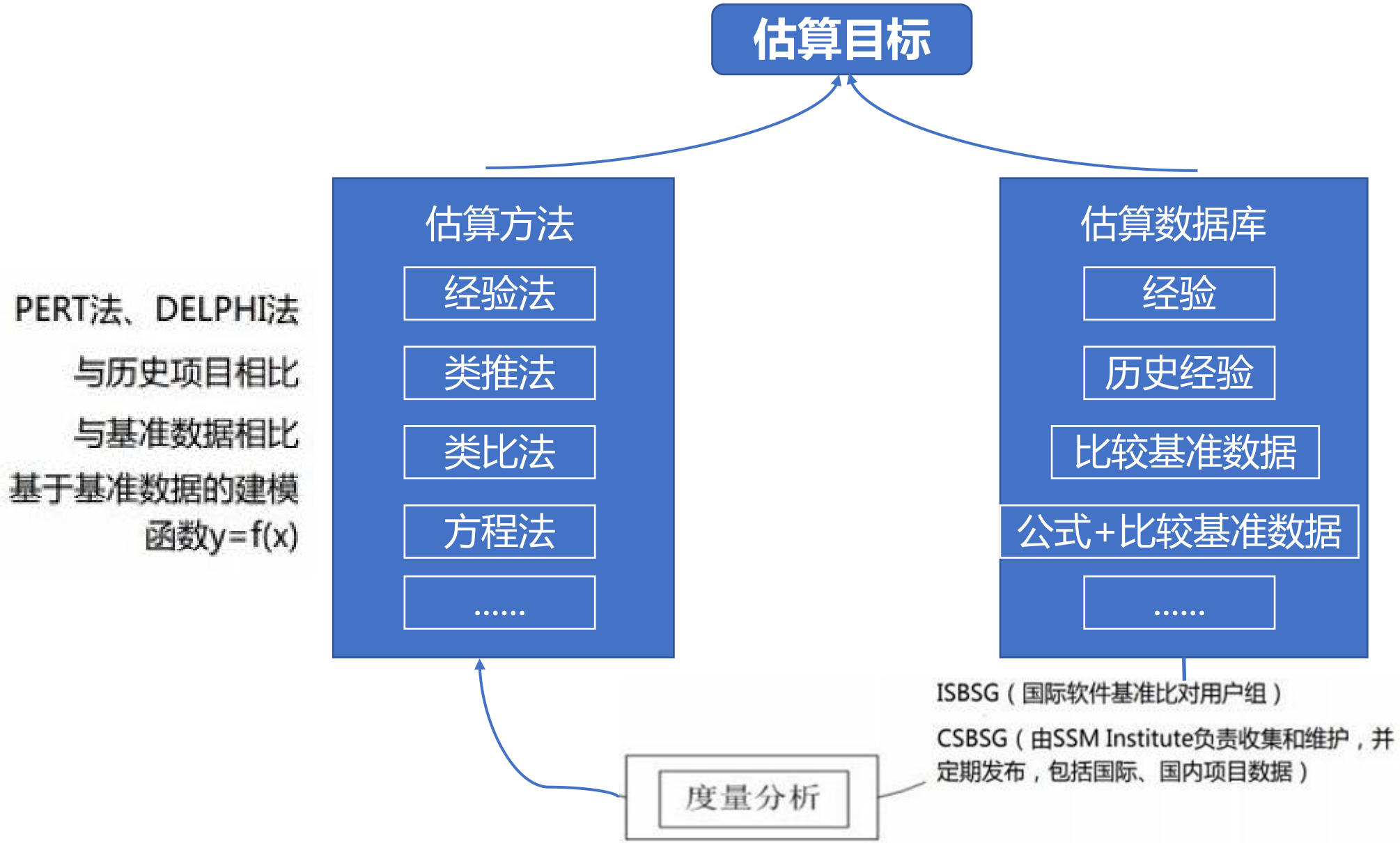
- 人天：需要1个人工作多少天
- 人月：需要1个人工作多少月
- 人年：需要1个人工作多少年

# 国标《软件开发成本度量规范》— 软件研发成本构成



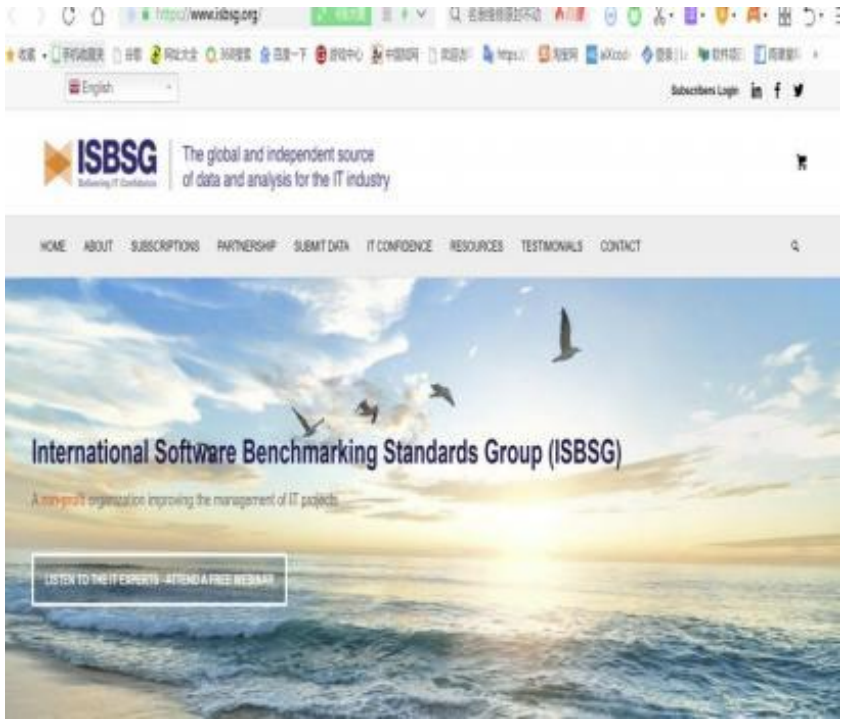

**直接人力成本估算是软件研发成本估算中的难点！**

# 软 件 估 算 思 路


估算目标
估算方法
经验法
类推法
类比法
方程法
……
估算数据库
经验
历史经验
比较基准数据
公式+比较基准数据
……
PERT法、DELPHI法
与历史项目相比
与基准数据相比
基于基准数据的建模
函数y=f(x)
ISBSG（国际软件基准比对用户组）
CSBSG（由SSM Institute负责收集和维护，并定期发布，包括国际、国内项目数据）
度量分析

# 软件行业基准数据比对平台

## ISBSG： 国际软件基准比对用户组




## CSBSG中国IT行业权威基准数据平台




https://www.bscea.org/html/rjjzsj/ndsj/2025/1029/3145.html

| | 25%分位数 | 中值 | 75%分位数 |
|---|---|---|---|
| 规模 | 1000功能点 | | |
| 工作量 | 22.64人月 | 42.04人月 | 75.79人月 |
| 价格 | 33.96万元 | 63.07万元 | 113.69万元 |

表 4.1 软件开发生产率基准数据明细

| 软件开发生产率详细信息（单位：人时/功能点） | | | | |
|---|---|---|---|---|
| P10 | P25 | P50 | P75 | P90 |
| 2.20 | 3.77 | 6.72 | 12.28 | 17.35 |

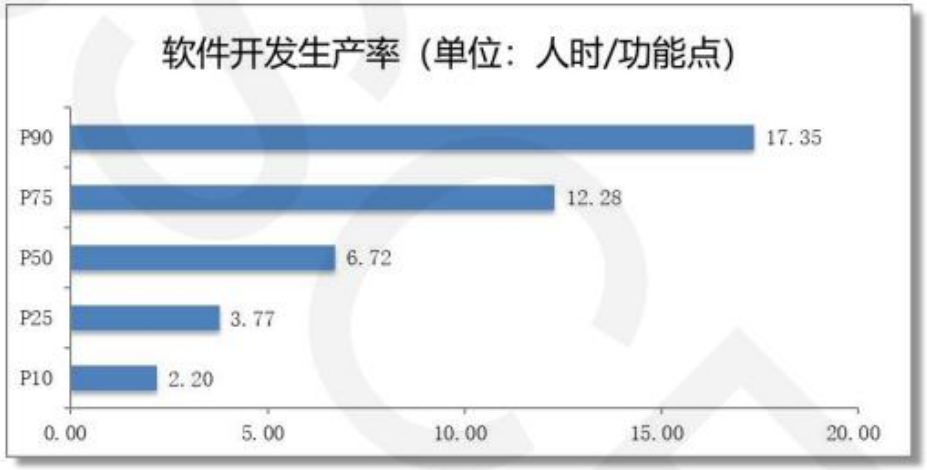



图 4.1 软件开发生产率




图 6.1 全行业软件开发生产率变化趋势

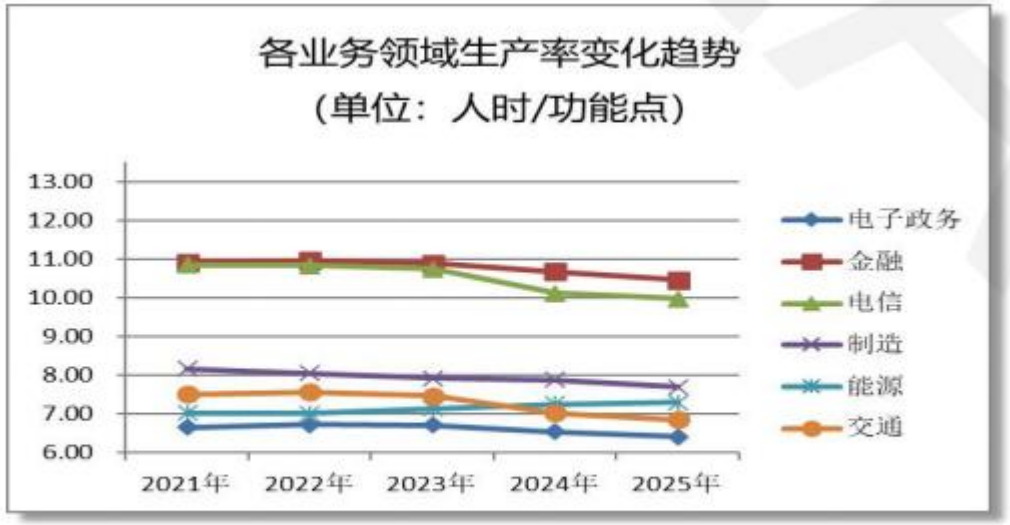
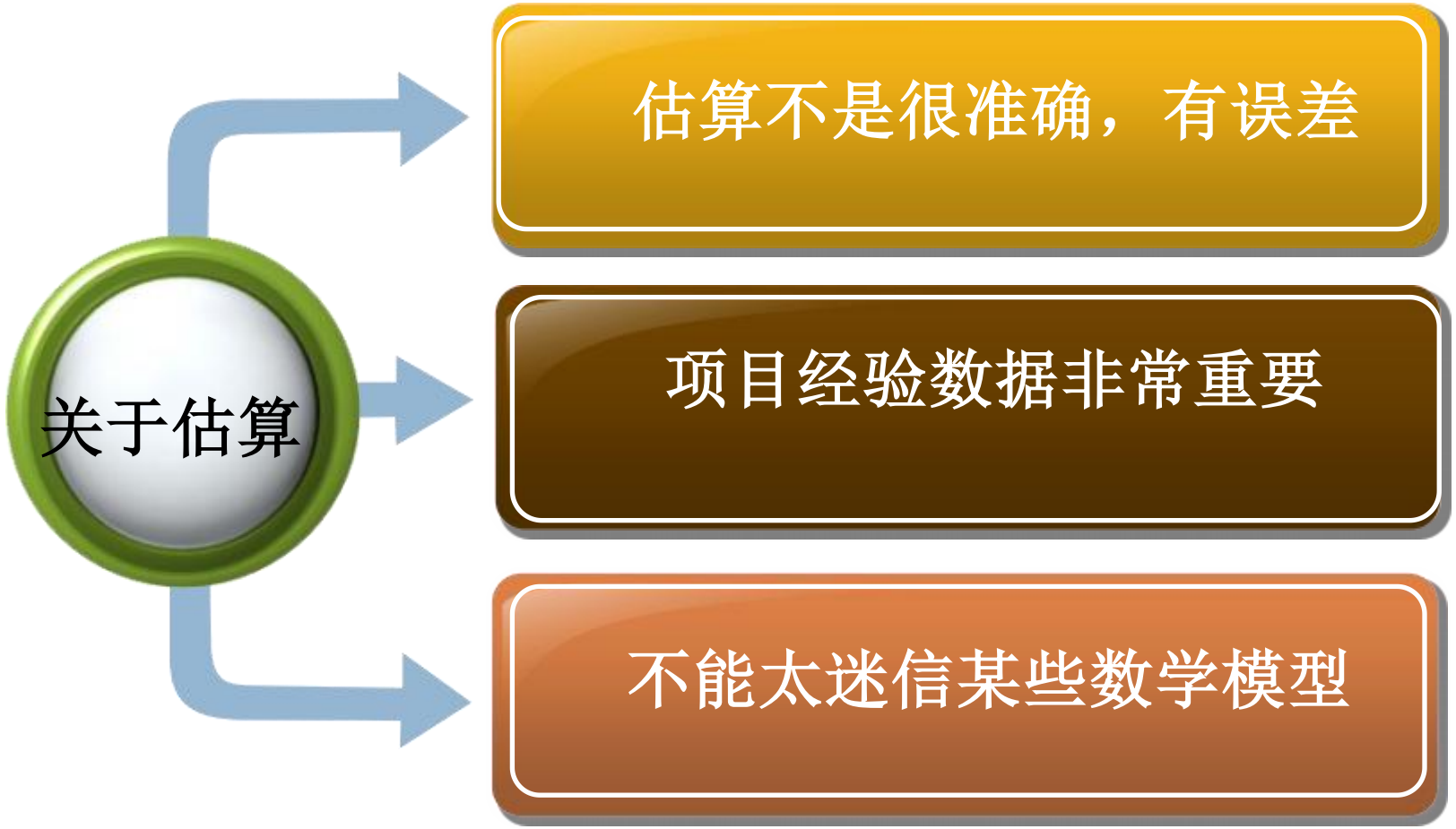



图 6.2 各业务领域软件开发生产率变化趋势

# 成本估算非常重要



# 本 章 要 点

一 估算过程概念

二 传统估算方法

三 敏捷估算方法

四 AI开发范式下的成本估算

五 成本预算

六 案例分析

# 传 统 估 算 方 法



1. 代码行估算法
2. 功能点估算法
3. 用例点估算法
4. 类比(自顶向下)估算法
5. 自下而上估算法
6. 三点估算法
7. 参数估算法
8. 专家估算法

# 1.代码行估算方法

## 从软件程序量的角度定义项目规模

与具体的编程语言有关
分解足够详细
有一定的经验数据

# 1.代码行估算方法

## 主要优点

代码是所有软件开发项目都有的“产品”，而且很容易计算代码行数。

## 主要缺陷

1. 对代码行没有公认的可接受的标准定义；
2. 代码行数量依赖于所用的编程语言和个人的编程风格；
3. 在项目早期，需求不稳定、设计不成熟、实现不确定的情况下很难准确地估算代码量；
4. 代码行强调编码的工作量，只是项目实现阶段的一部分。

## 例：汉诺塔问题的求解-递归的方法

python

```python
def hanoi(n, source, target, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    hanoi(n-1, source, auxiliary, target)
    print(f"Move disk {n} from {source} to {target}")
    hanoi(n-1, auxiliary, target, source)

# 示例：移动3个盘子从柱子'A'到柱子'C'，使用柱子'B'作为辅助
hanoi(3, 'A', 'C', 'B')
```

# 1.代码行估算方法

## 例：汉诺塔问题的求解-循环的方法

python

```python
def hanoi_iterative(n, source, target, auxiliary):
    def move_disk(disk, source, target):
        print(f"Move disk {disk} from {source} to {target}")

    def get_direction(n):
        return 1 if n % 2 == 1 else -1

    moves = 2**n - 1
    direction = get_direction(n)

    for i in range(1, moves + 1):
        if i % 3 == 1:
            move_disk(i % 2 + 1, source, target if direction == 1 else auxiliary)
        elif i % 3 == 2:
            move_disk(i % 2 + 1, source, auxiliary if direction == 1 else target)
        else:
            move_disk(i % 2 + 1, auxiliary, source)
        direction = -direction

# 示例：移动3个盘子从柱子'A'到柱子'C'，使用柱子'B'作为辅助
hanoi_iterative(3, 'A', 'C', 'B')
```

# 传 统 估 算 方 法



1. 代码行估算法
2. 功能点估算法
3. 用例点估算法
4. 类比(自顶向下)估算法
5. 自下而上估算法
6. 三点估算法
7. 参数估算法
8. 专家估算法

# 2.功能点估算方法

**功能点**（Function Points）：是衡量软件规模的一个标准度量单元，在软件度量中广泛应用，从用户角度将系统分解成更小的模块，便于理解和分析，为项目范围、工作量、资源、时间等因素进行估算提供了依据。

**功能点估算特点：**

- 功能点估算法常在项目开始或项目需求基本明确时使用，这时进行估算其结果的准确性比较高。
- 使用功能点估算法无需懂得软件使用何种开发技术。
- 功能点估算法是以用户为角度进行估算。
- 通过一些行业标准或企业自身度量的分析。

# 2.功能点估算方法

**评估组织**:

**IFPUG （国际功能点用户组织）**

NESMA （荷兰软件度量协会）

ISBSG （国际软件基准比对标准化组织）

KOSMA （韩国软件度量协会）

IPASEC （日本软件工程中心）

相关国际标准:

《基准比对标准》、《推荐度量元集》、《用户组术语集》、《软件度量纲要》、《功能点计数手册 （中文版） 》、《中国软件生产力报告》、《韩国软件估算国家标准》、《日本软件度量白皮书》。

# IFPUG功能点估算

- IFPUG(国际功能点用户组织)功能点
- 由IBM公司的Alan Albrecht提出，也称为Albrecht功能点
- 适用于信息系统

功能点公式

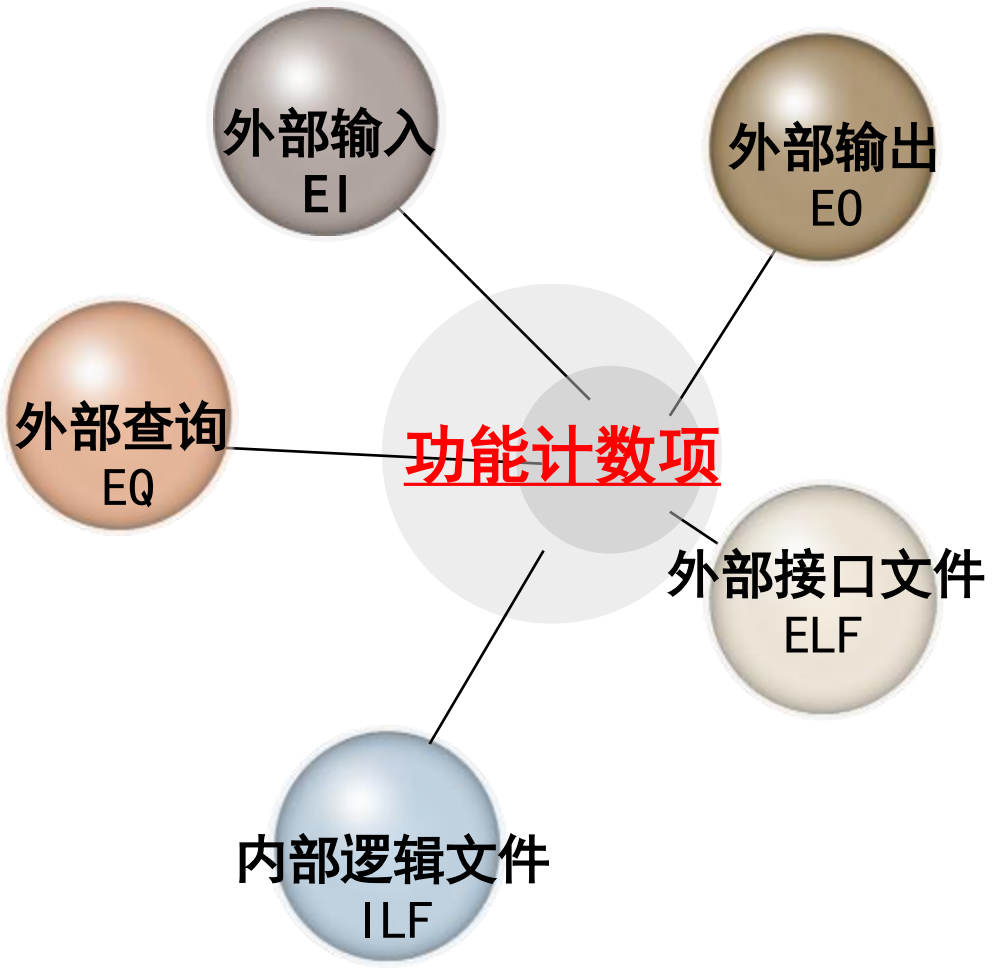
FP =UFC*TCF

- UFC：未调整功能点计数
- TCF：技术复杂度因子

# 2.功能点估算方法

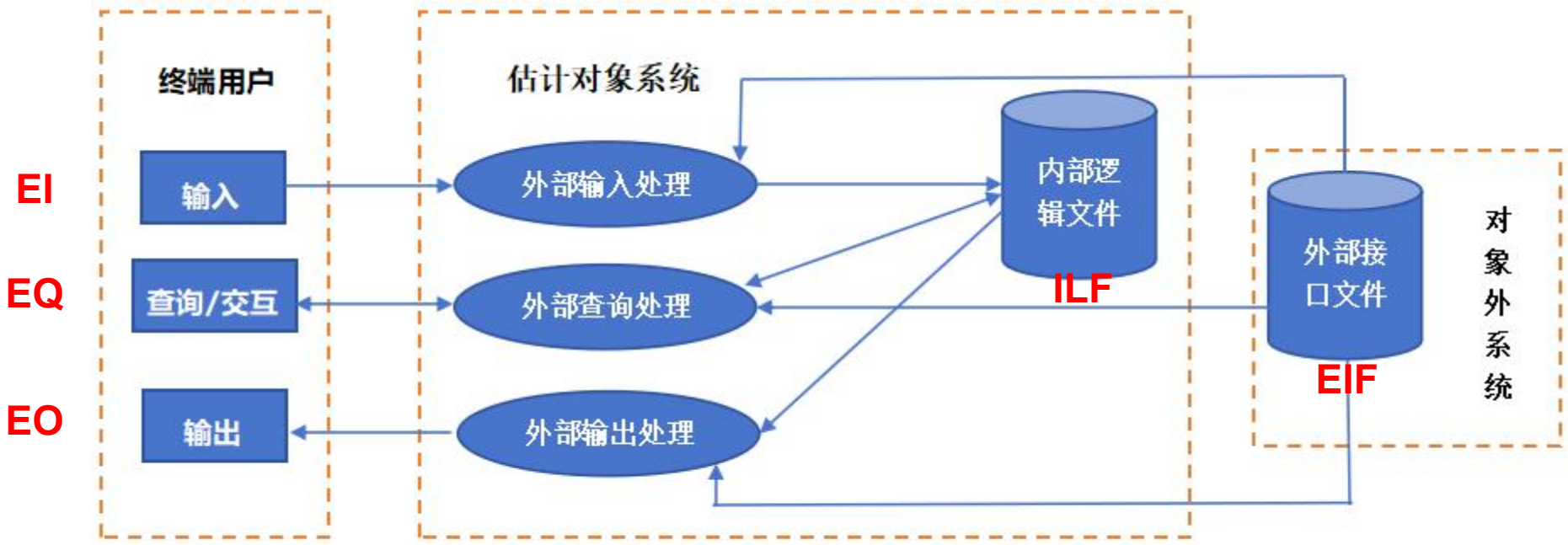
- **UFC(Unadjusted Function Component)未调整功能点计数，来自于五项内容：**

① ILF：Internal Logical File 内部逻辑文件
② EIF: External Interface File 外部接口文件
③ EI: External Input 外部输入
④ EO: External Output 外部输出
⑤ EQ: External Inquiry 外部查询




- 其中，ILF和EIF属于数据类型的功能点，EI、EO、EQ属于人机交互事务类型的功能点。

# 2.功能点估算方法




构成软件的 5 种要素及关系图

# 内部逻辑文件ILF与外部接口文件EIF

- **ILF内部逻辑文件**
  内部逻辑文件是指一组以用户角度识别的、在应用程序边界内被维护的逻辑相关数据或控制信息。ILF的主要目的是通过应用程序的一个或多个基本处理过程来维护数据。
- **EIF外部接口文件**
  外部接口文件是指一组在应用程序边界内被查询， 但在其他应用程序中被维护的、以用户角度来识别的、逻辑上相关的数据。因此，一个应用程序中的EIF必然是其他应用程序中的ILF。EIF的主要目的是为边界内的应用程序提供一个或多个通过基础操作过程来引用的一组数据或信息。
- EIF所遵循的规则：
  ① 从用户角度出发识别的一组逻辑数据。
  ② 这组数据是在应用程序外部，并被应用程序引用的。
  ③ 计算功能点的这个应用程序并不维护该EIF。
  ④ 这组数据是作为另一个应用程序中的ILF被维护的。

**ILF和EIF的复杂性**取决于RET和DET的数量。

- DET（Data element type）是一个以用户角度识别的、非重复的、有业务逻辑意义的字段。
- RET（Record element type）是指一个EIF/ILF中用户可以识别的DET的集合。如果把DET简单理解为字段的话，那RET就可以简单理解为数据库中的表。

# 2.功能点估算方法

- 计算RET的规则为以下两点：
  - 在一个ILF/EIF中每一个可选或必选的集合都被计算为一个RET。
  - 如果一个ILF/EIF没有子集合， 则ILF/EIF被计算为一个RET。
- 例如： 在外贸订单系统中添加一个订单时会保存“订单信息、客户的ID、部门的ID”。那么订单系统ILF中的RET为：

① 订单信息 （必选的）

② 客户信息 （必选的）

③ 部门信息 （可选的）

因此ILF中RET的个数为3个。

**ILF和EIF的复杂性**取决于RET和DET的数量。

- DET计算的规则如下：
  - 通过一个基本处理过程的执行，对ILF进行维护，或从ILF/EIF中返回一个特定的、用户可识别的、非重复的字段，那么每个这样的字段算一个DET。
    - 例如：添加一个外贸订单时需要保存“订单号码、订单日期、地址、邮编”，那么对于ILF订单来说它的DET就是4个。
    - 再如：保存订单时还会保存订单的明细。订单的明细往往作为一个子表进行保存，那么“订单号码”在主表和子表中都同时存在（主外键）。但以用户角度来识别时，存盘操作是一个最小的单位，那么订单号码只能算做一个DET。
  - 当两个应用程序维护和/或引用相同的ILF/EIF，但是每个应用程序分别维护/引用它们相应的DET时，这些DET在这两个应用程序的维护/引用中将单独计算。
    - 例如，一个应用程序的两个基本处理过程都需要使用到“地址”的信息，地址信息又可以细分为“国家、城市、街道、邮编”。那么对于其中一个基本处理过程来说，它将整个地址信息作为一个整体进行处理，只算一个DET；另外一个基本处理过程使用每个地址的详细信息，那么DET就是4个。

**ILF和EIF的复杂性**取决于RET和DET的数量。

ILF/EIF复杂度的矩阵

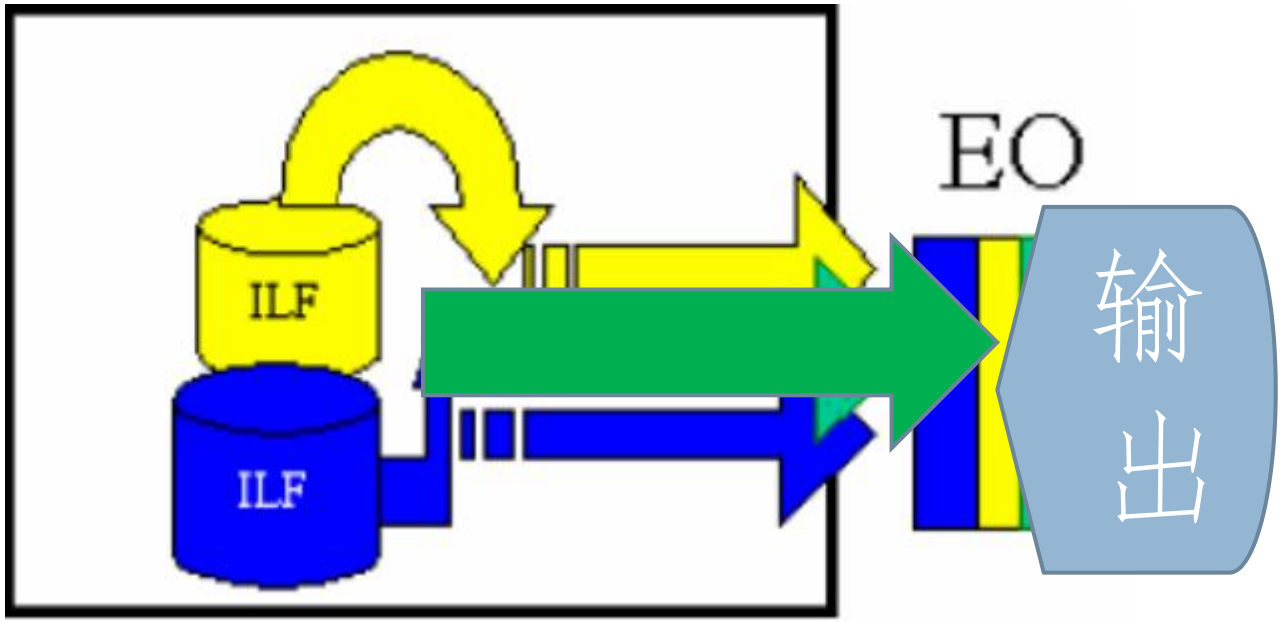
| | 1~19个DET | 20~50个DET | 超过51个DET |
|---|---|---|---|
| 1个RET | 低 | 低 | 中等 |
| 2~5个RET | 低 | 中等 | 高 |
| 6个以上RET | 中等 | 高 | 高 |

# 外部输入(External Inputs:EI)

- 给软件提供面向应用的数据的项（如屏幕、表单、对话框、控件，文件等）；在这个过程中，数据穿越外部边界进入到系统内部。
- EI是处理来自应用程序边界外部的一组数据输入，它的主要目的是维护一个或多个ILF，以及/或者更改系统的行为。常用的增、改、删操作（没有查）都属于外部输入。

# 外部输出(External Outputs:EO)

向用户提供(**经过处理的**)面向应用的信息，例如，报表和出错信息等。






# 外部查询(External Inquiry:EQ)

外部查询是一个输入引出一个即时的简单输出，没有处理过程。

- EQ是向应用程序边界外发送数据基本处理的过程。其主要目的是从ILF或EIF中通过恢复数据信息来向用户呈现。该处理逻辑**不包括任何数学公式**或计算方法，也不会生成任何派生数据。EQ不会维护任何一个ILF，也不会改变应用程序的系统行为。
- EO是输送数据到应用程序边界外部的过程，它的主要目的是通过逻辑处理过程向用户呈现信息。该处理过程**必须包含至少一个数学公式或计算方法**，或生成派生数据，一个EO也可以维护一个或多个ILF。
- EO和EQ的主要目的都是通过基本操作过程展现数据给用户。

## EI、EO、EQ的主要目的

| 目的 | EI | EO | EQ |
|---|---|---|---|
| 改变应用程序的属性或行为 | 主要目的 | 次要目的 | 不允许 |
| 维护一个或多个ILF | 主要目的 | 次要目的 | 不允许 |
| 显示信息给用户 | 次要目的 | 主要目的 | 主要目的 |

## EI、EO、EQ的主要行为

| 行为 | EI | EO | EQ |
|---|---|---|---|
| 数学公式或计算被执行 | 可以 | 至少选择一次 | 不可以 |
| 至少一个ILF被修改 | 至少选择一次 | 至少选择一次 | 不可以 |
| 至少一个ILF或EIF被引用 | 可选 | 可选 | 必选 |
| 数据被重新恢复 | 可选 | 可选 | 必选 |
| 派生数据被创建 | 可选 | 至少选择一次 | 可选 |
| 应用程序的行为或属性被修改 | 至少选择一次 | 至少选择一次 | 可选 |
| 准备或呈现信息到系统边界外 | 可选 | 必选 | 必选 |
| 接受进入系统边界内的数据的能力 | 必须 | 可选 | 可选 |

# EI、EQ和EO的技术复杂性计算

- 复杂性取决于FTRs和DETs的数量。FTR(File Type Referenced)是被一个事物读取或维护的ILF，或者是被一个事物读取的EIF。
- **EI中识别FTR规则**
  - 每一个ILF应该算做一个FTR。
  - 通过EI读取的每个ILF或EIF都应该计算为一个FTR。
  - 既被EI维护又被读取的ILF仅计算为一个FTR。
- **EI中识别DET规则**
  - 在EI的过程中，以用户角度识别的、通过应用系统边界输入系统内部的非重复字段，应算作一个DET。
  - 在EI的过程中，只要没有通过系统边界输入，即使它存在于系统内的一个ILF中，也不能算为一个DET。例如：外贸订单系统中，订单的金额是被单价和数量自动计算的，那么金额是没有通过系统边界输入的，因此在EI操作中就不应该算做一个DET。
  - 在应用程序的EI操作时，系统提示的错误信息或完成操作的信息，应该被分别计算为一个DET。例1：在网站注册用户信息时，由于输入错误系统会显示提示信息，那么这些提示信息应该被逐个计算为一个DET。例2：当EI操作完成时系统提示并显示出来的信息，应该被计算为一个DET。
  - 在EI操作中，如果遇到主键外的字段，应该算作一个DET。

- **EO和EQ计算FTR的规则**

1. 通用规则：每个在EO/EQ处理过程中读取的ILF和EIF算一个FTR
2. EO额外的FTR计算规则：
   - 在EO处理过程中每个被维护的ILF算一个FTR
   - 在EO处理过程中既被读取又被维护的ILF算一个FTR

- **EO和EQ计算DET的通用规则**

– 用户可识别的非重复字段，进入应用边界并指明处理什么、何时处理或处理方式，并且由EO/EQ返回或产生，那么这样的每个字段算一个DET。

例如：报表中的每个字段都是一个DET。

– 在应用边界内以用户角度识别的非重复字段算一个DET。

例如：在报表中起到解释或备注作用的文字信息，不管是一个字、一个词或一段话，都当作一个DET。

再如：某种编号或日期，即使它被物理存储在不同字段中，但从用户角度看是一个整体的信息，因此被算作一个DET。还有：在饼图中百分比和分类算作不同的DET。

# ILF和EIF的定级矩阵

**内部逻辑文件ILF或者外部接口文件EIF的定级表**

| 记录元素类型（RET's） | 数据元素（Data Elements） | | |
|---|---|---|---|
| | 1-19 | 20-50 | >50 |
| 1 | 低 | 低 | 中 |
| 2-5 | 低 | 中 | 高 |
| >5 | 中 | 高 | 高 |

**内部逻辑文件ILF或者外部接口文件EIF的定级取值表**

| 级数（Rating） | 值 | |
|---|---|---|
| | ILF | EIF |
| 低 | 7 | 5 |
| 中 | 10 | 7 |
| 高 | 15 | 10 |

# 事务组件定级矩阵

## 外部输入EI的定级表

| 引用的文件类型个数（FTR's） | 数据元素（Data Elements） | | |
|---|---|---|---|
| | 1-4 | 5-15 | >15 |
| 0-1 | 低 | 低 | 低 |
| 2 | 低 | 中 | 高 |
| >=3 | 中 | 高 | 高 |

## 外部输出EO和外部查询EQ共用的定级表

| 引用的文件类型个数（FTR's） | 数据元素（Data Elements） | | |
|---|---|---|---|
| | 1-5 | 6-19 | >19 |
| 0-1 | 低 | 低 | 中 |
| 2-3 | 低 | 中 | 高 |
| >3 | 中 | 高 | 高 |

## 外部输入EI、外部输出EO和外部查询EQ的定级取值表

| 级数（Rating） | 值 | | |
|---|---|---|---|
| | EO | EQ | EI |
| 低 | 4 | 3 | 3 |
| 中 | 5 | 4 | 4 |
| 高 | 7 | 6 | 6 |

# UFC-未调整功能点计数

功能计数项的复杂度等级

| | 复杂度权重因素 | | |
|---|---|---|---|
| 项 | 简单（低） | 一般（中） | 复杂（高） |
| 外部输入EI | 3 | 4 | 6 |
| 外部输出EO | 4 | 5 | 7 |
| 外部查询EQ | 3 | 4 | 6 |
| 外部接口文件EIF | 5 | 7 | 10 |
| 内部逻辑文件ILF | 7 | 10 | 15 |

# 例：利用功能点估算法计算员工信息管理系统业务复杂度

- 在员工管理系统中添加一个员工资料，假设会使用到员工的一般信息、教育情况、工作经历和家属信息。员工隶属于某个部门，在本系统中会有一个对部门进行 维护的功能。员工的工资则由另外一个财务系统提供。

- 假设员工基本信息如下：
  - 员工ID（标签控件）
  - 员工姓名
  - 性别
  - 生日
  - 婚否
  - 所属部门ID（标签控件）

  - 所属部门名称
  - 受教育的时间
  - 学校名称
  - 所学专业
  * 工作时间
  * 工作单位
  * 工作部门
  * 工作职务
  - 亲属的姓名
  - 之间关系
  - 亲属联系方式
  - 亲属工作单位

- 假设部门信息如下：
  - 部门ID（标签控件）
  - 部门名称
- 假设工资表信息如下：
  - 员工ID（标签控件）
  - 员工姓名
  - 金额
  - 单位

- 本范例识别出来ILF和EIF功能点个数如下表所示。

| ILF内部逻辑文件 | RET | DET个数 | 复杂度 | 未调整的FP个数 |
|---|---|---|---|---|
| 员工信息 | 员工基本信息、受教育情况、工作经历、亲属信息，共4个表。 | 18 | 低 | 7 |
| 部门信息 | 部门基本信息，共1个。 | 2 | 低 | 7 |

| EIF外部接口文件 | RET | DET个数 | 复杂度 | 未调整的FP个数 |
|---|---|---|---|---|
| 工资表 | 员工基本信息 、工资信息， 共2个。 | 4 | 低 | 5 |

合计： 19个

# 2.功能点估算方法

**EI、EQ和EO的功能点数**

| EI | FTR | DET个数 | 复杂度 | 未调整的FP个数 |
|---|---|---|---|---|
| 添加员工信息 | 员工、部门、工资表 | 18个<br>员工信息的两个标签控件内容不是用户输入的，因此不算，共16个；部门信息与员工信息中的部门字段重复，因此一个都不算；工资表中的员工ID和名称不能重复，因此只能算金额和单位，所以共2个。 | 高 | 6 |
| 修改员工信息 | 员工、部门、工资表 | 18个<br>同上 | 高 | 6 |
| 删除员工信息 | 员工、部门、工资表 | 1个<br>员工ID | 中等 | 4 |
| 添加部门信息 | 部门 | 1个<br>一个标签控件的内容不是用户输入的， 因此不算 | 低 | 3 |
| 修改部门信息 | 部门 | 1个<br>一个标签控件的内容不是用户输入的， 因此不算 | 低 | 3 |
| 删除部门信息 | 部门 | 1个部门ID | 低 | 3 |

合计25个

# EI、EQ和EO的功能点数

| EQ | FTR | DET个数 | 复杂度 | 未调整的FP个数 |
|---|---|---|---|---|
| 查询员工信息 | 员工、部门、工资表 | 20 | 高 | 6 |
| 查询部门信息 | 部门 | 2 | 低 | 3 |

合计： 9个

| EO | FTR | DET个数 | 复杂度 | 未调整的FP个数 |
|---|---|---|---|---|
| 统计员工年薪 | 员工、工资表 | 员工ID、员工名称、年份、年薪、单位共5个 | 低 | 4 |

合计： 4个

则UCF=19+25+9+4=57

# 计算调整因子

- 用功能点估算法计算软件项目功能点时会用到调整因子（或称调整系数）。功能点的调整系数是通过通用系统特性及其影响程度来评定的，对每个常规系统特性的评估由其影响程度（DI）而定，分为0-5级：
  - 0 毫无影响
  - 1 偶然影响
  - 2 适度影响
  - 3 一般影响
  - 4 重要影响
  - 5 强烈影响
- 然后依次对以下14个系统常规特性进行打分，并带入以下计算公式算出功能点的调整因子。

  Technical Complexity Factor(TCF)= 0.65+( sum of (DI) * 0.01 )

# 2.功能点估算方法

| 系统特性 | 分数 |
| --- | --- |
| 数据通讯 | 3 |
| 分布式数据处理 | 2 |
| 性能 | 0 |
| 高强度配置 | 0 |
| 交易速度 | 0 |
| 在线数据输入 | 5 |
| 最终用户效率 | 2 |
| 在线更新 | 3 |
| 复杂处理 | 0 |
| 可复用性 | 3 |
| 易安装性 | 0 |
| 易操作性 | 0 |
| 多场地 | 0 |
| 支持变更 | 1 |

合计19

技术复杂度因子TCF=0.65+0.01(sum(Fi))：

Fi:0-5, TCF区间:0.65-1.35

- 本例中TCF = 0.65+0.01*19=0.84
- 考虑到前述UFC=19+25+9+4=57，则最终调整后的功能点数量为：

  FP =UFC*TCF=57*0.84 = 47.88个
- 如果项目的生产率PF=20工时/功能点，则项目的工作量是20*47.88=957.6

# 功能点与代码行的转换

| 编程语言 | SLOC/FP | 编程语言 | SLOC/FP |
|---|---|---|---|
| Assembly | 320 | Object languages | 30 |
| **C** | 128 | Smalltalk | 22 |
| Cobol | 105 | generation languages | 20 |
| Fortran | 105 | Powerbuilder | 16 |
| Pascal | 90 | Koodigenerators | 15 |
| Ada | 70 | SQL | 12 |
| **C++** | 64 | Spread sheat pogram | 6 |
| Ada 95 | 53 | Graphical language | 4 |
| Visual Basic | 32 | **Java** | 48 |
| **C#** | 36 | VB | 29 |
| Delphi | 18 | 代码生成器 | 15 |

# 传 统 估 算 方 法



1. 代码行估算法
2. 功能点估算法
3. 用例点估算法
4. 类比(自顶向下)估算法
5. 自下而上估算法
6. 三点估算法
7. 参数估算法
8. 专家估算法

# 3.用例点估算方法

- 在软件开发中，用例和功能虽然紧密相关，但它们并不是一回事。
- 功能指的是软件系统所具有的能力或者可以执行的任务。它是从软件产品的角度出发，描述了软件能做什么，比如“用户注册”、“数据备份”、“在线支付”等。功能是面向结果的，它描述了系统提供的具体服务。
- 用例则是一种在系统分析和设计过程中使用的工具，用来描述系统如何响应外部请求。它通常包括一系列步骤，描述了一个角色（用户或其他系统）与系统交互以完成某个目标的过程。用例不仅描述了功能，还描述了实现该功能所需的步骤、条件以及可能的替代路径。

# 3.用例点估算方法

- 例如，一个在线购物系统中的一个功能可能是“添加商品到购物车”，而对应的用例可能会详细描述如下：

- 用例名称：添加商品到购物车；主要参与者：顾客；前置条件：顾客已登录，商品库存充足
- 基本流程：

  顾客浏览商品列表。

  顾客选择一个商品。

  顾客点击“添加到购物车”按钮。

  系统将商品添加到顾客的购物车中。

  系统显示购物车更新后的商品列表。
- 替代流程：

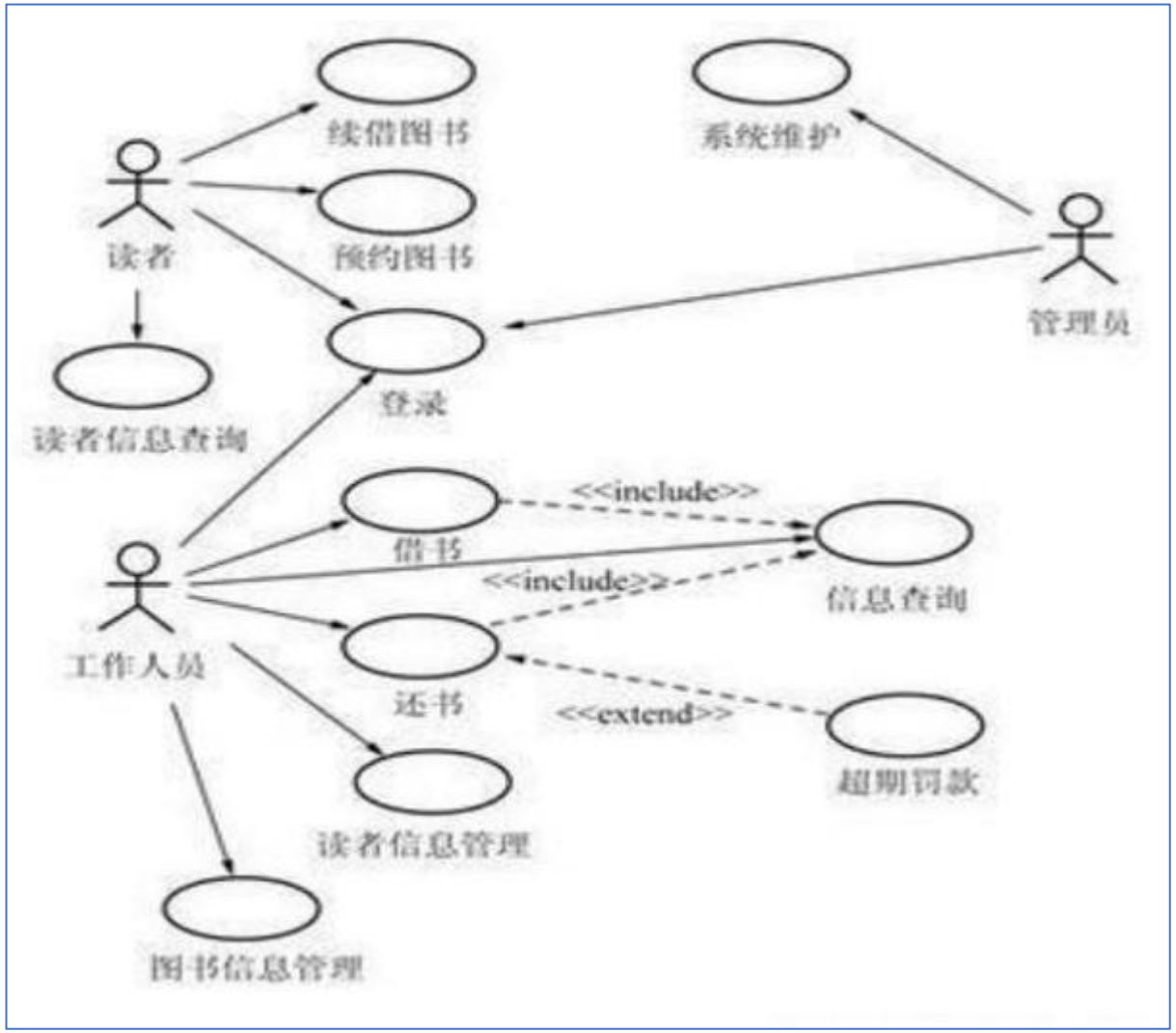
  如果商品库存不足，系统显示错误消息

  如果顾客尚未登录，系统提示顾客登录

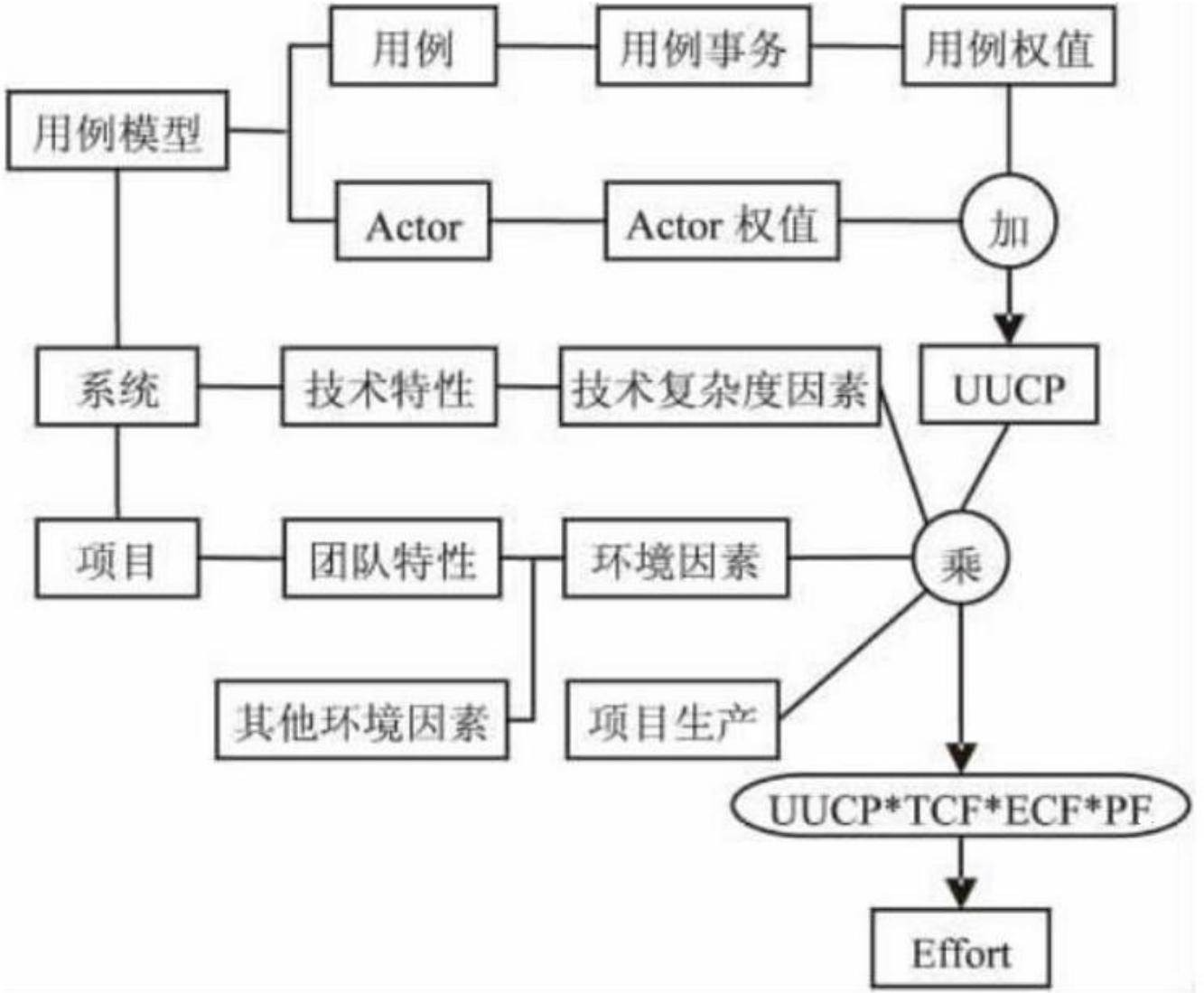
- 用例帮助开发团队理解需求，确保所有的交互都被考虑到，并且在开发过程中得到实现。而功能则是产品宣传、用户手册等文档中常用的术语，它更侧重于告诉用户软件能做什么。

# 3.用例点估算方法

## 借书系统



# 用例点估算模型


用例
用例事务
用例权值
用例模型
Actor
Actor 权值
加
系统
技术特性
技术复杂度因素
UUCP
项目
团队特性
环境因素
乘
其他环境因素
项目生产
UUCP*TCF*ECF*PF
Effort

# 用例点估算方法的基本步骤

1. 计算未调整的角色权值UAW;
2. 计算未调整的用例权值UUCW;
3. 计算未调整的用例点UUCP（=UAW+UUCW）；
4. 计算技术和环境因子TCF;
5. 计算调整的用例点UCP；
6. 计算工作量( 人时) 。

# 1、计算未调整的角色权值UAW

$$UAW = \sum aWeight(c) \times aCardinality(c)$$

角色数量与对应权重之积的和

| 角色类型 | 说明 | 权值 |
| --- | --- | --- |
| 简单角色 (Simple) | 通过标准化接口与系统交互（如API、第三方服务）。 | 1 |
| 中等角色 (Average) | 通过交互协议（如HTTP、TCP）或简单GUI与系统交互（如用户通过表单提交数据）。 | 2 |
| 复杂角色 (Complex) | 通过复杂协议或图形界面交互（如用户需多步骤操作、实时交互的GUI）。 | 3 |

## 2、计算未调整的用例权值UUCW

$$UUCW = \sum uWeight(c) \times uCardinality(c)$$

用例数量与对应权重之积的和

| 序号 | 复杂度级别 | 事务/场景个数 | 权值 |
|---|---|---|---|
| 1 | simple | 1—3 | 5 |
| 2 | average | 4—7 | 10 |
| 3 | complex | >7 | 15 |

# 3、计算未调整的用例点UUCP

## UUCP=UAW+UUCW: 例如

| 序号 | 复杂度级别 | 权值 | 参与角色数 | UAWi |
|---|---|---|---|---|
| 1 | simple | 1 | 2 | 2 |
| 2 | average | 2 | 4 | 8 |
| 3 | complex | 3 | 5 | 15 |

| 序号 | 复杂度级别 | 权值 | 用例数 | UUCWi |
|---|---|---|---|---|
| 1 | simple | 5 | 5 | 25 |
| 2 | average | 10 | 2 | 20 |
| 3 | complex | 15 | 3 | 45 |

UAW=2+8+15=25

UUCW=25+20+45=90

UUCP=UAW+UUCW=115

# 4、计算技术复杂度因子TCF

$$TCF=0.6+(0.01\times\sum_{i=1}^{13}TCF_Weight_i\times Value_i)$$

| 序号 | 技术因子 | 说明 | 权值 |
|---|---|---|---|
| 1 | TCF1 | 分布式系统 | 2.0 |
| 2 | TCF2 | 性能要求 | 1.0 |
| 3 | TCF3 | 最终用户使用效率 | 1.0 |
| 4 | TCF4 | 内部处理复杂度 | 1.0 |
| 5 | TCF5 | 复用程度 | 1.0 |
| 6 | TCF6 | 易于安装 | 0.5 |
| 7 | TCF7 | 系统易于使用 | 0.5 |
| 8 | TCF8 | 可 移 植 性 | 2.0 |
| 9 | TCF9 | 系统易于修改 | 1.0 |
| 10 | TCF10 | 并发性 | 1.0 |
| 11 | TCF11 | 安全功能特性 | 1.0 |
| 12 | TCF12 | 为第三方系统提供直接系统访问 | 1.0 |
| 13 | TCF13 | 特殊的用户培训设施 | 1.0 |

根据项目的具体情况，为每项技术因子在0-5区间赋值，再乘以对应的权值后加总乘以0.01再加0.6，可得TCF值。

TCF=0.6+0.01×(2.0×3+1.0×5+1.0×3+1.0×5+1.0×0+0.5×3+0.5×5+2.0×3+1.0×5+1.0×3+1.0×5+1.0×0+1.0×0)=1.02

# 4、计算环境因子ECF

$$=1.4+(-0.03\times\sum_{i=1}^{8}ECF_Weight_i\times Value_i)$$

| 序号 | 环境因子 | 说明 | 权值 |
|---|---|---|---|
| 1 | ECF1 | UML精通程度 | 1.5 |
| 2 | ECF2 | 系统应用经验 | 0.5 |
| 3 | ECF3 | 面向对象经验 | 1.0 |
| 4 | ECF4 | 系统分析员能力 | 0.5 |
| 5 | ECF5 | 团队士气 | 1.0 |
| 6 | ECF6 | 需求稳定度 | 2.0 |
| 7 | ECF7 | 兼职人员比例高低 | 1.0 |
| 8 | ECF8 | 编程语言难易程度 | 1.0 |

同TCF的计算，根据项目的具体情况，为每项技术因子在0-5区间赋值，再乘以对应的权值后加总减去0.03再加1.4，可得ECF值。

ECF=1.4-0.03×(1.5×3+0.5×3+1.0×3+0.5×5+1.0×3+2.0×3+1.0×0+1.0×0)=0.785

## 5、计算调整的用例点UCP

$$UCP = UUCP \times TCF \times ECF$$
$$= 115 \times 1.02 \times 0.785 = 92$$

## 6、计算工作量

$$Effort = UCP \times PF$$

如果生产力系数PF = 20工时/用例点

则：Effort =UCP × PF = 92×20 = 1840h=230人天

# 传 统 估 算 方 法

1. 代码行估算法
2. 功能点估算法
3. 用例点估算法
4. 类比(自顶向下)估算法
5. 自下而上估算法
6. 三点估算法
7. 参数估算法
8. 专家估算法

# 传统估算方法-类比估算-定义

- 估算人员根据以往的完成类似项目所消耗的总成本（或工作量），来推算将要开发的软件的总成本（或工作量）。
- 是一种自上而下的估算形式

# 类比估算—使用情况

- 有类似的历史项目数据
- 信息不足（例如市场招标）的时候
- 要求不是非常精确估算的时候

# 例：一个待估算工作量的项目$P_0$和已完成的项目$P_1$,$P_2$

| Project | Project type | Programming language | Team size | Project size | Effort |
|---|---|---|---|---|---|
| $P_0$ | MIS | C | 9 | 180 | ? |
| $P_1$ | MIS | C | 11 | 200 | 1000 |
| $P_2$ | Real time | C | 10 | 175 | 900 |

- **标称属性（名词性）的邻近度量**

- 2个或多个状态， e.g., red, yellow, blue, green (二元属性的推广)
- 方法1：简单匹配
  - $m$：p个变量中匹配的个数, $p$：全部变量的个数

$$d(i,j)=\frac{p-m}{p}$$

- 方法2：使用一系列的二进制属性
  - 为M个名义状态的每一个产生一个新的二进制/二元属性

- **标称属性（名词性）的邻近度量**
  - 2个或多个状态, e.g., red, yellow, blue, green (二元属性的推广)
  - Method 1: 简单匹配
    - $m$: $p$个变量中匹配的个数，$p$: 全部变量的个数

$$d(i,j)=\frac{p-m}{p}$$

| 姓名 | 帽子颜色 | 上衣颜色 | 裤子颜色 | 鞋子颜色 | 围巾颜色 |
|---|---|---|---|---|---|
| 张三 | 红 | 蓝 | 蓝 | 绿 | 红 |
| 李四 | 红 | 蓝 | 蓝 | 红 | 粉 |
| 王五 | 黄 | 绿 | 蓝 | 绿 | 红 |
| 孙六 | 蓝 | 蓝 | 蓝 | 红 | 粉 |

→

$$\begin{bmatrix} 0, & 2/5, & 2/5, & 3/5 \\ 2/5, & 0, & 4/5, & 1/5 \\ 2/5, & 4/5, & 0, & 4/5 \\ 3/5, & 1/5, & 4/5, & 0 \end{bmatrix}$$

# 二进制属性的邻近度量

- 二进制数据的列联表

| Object *i* \ Object *j* | 1 | 0 | sum |
|---|---|---|---|
| 1 | $q$ | $r$ | $q+r$ |
| 0 | $s$ | $t$ | $s+t$ |
| sum | $q+s$ | $r+t$ | $p$ |

- 对称二元变量的距离侧度:

$$d(i,j)=\frac{r+s}{q+r+s+t}$$

- 不对称二元变量的距离侧度:

$$d(i,j)=\frac{r+s}{q+r+s}$$

- Jaccard系数(不对称二元变量的相似性侧度):

$$sim_{Jaccard}(i,j)=\frac{q}{q+r+s}$$

# 二进制属性的相异度量

| Name | Gender | Fever | Cough | Test-1 | Test-2 | Test-3 | Test-4 |
|---|---|---|---|---|---|---|---|
| Jack | M | Y | N | P | N | N | N |
| Mary | F | Y | N | P | N | P | N |
| Jim | M | Y | Y | N | N | N | N |

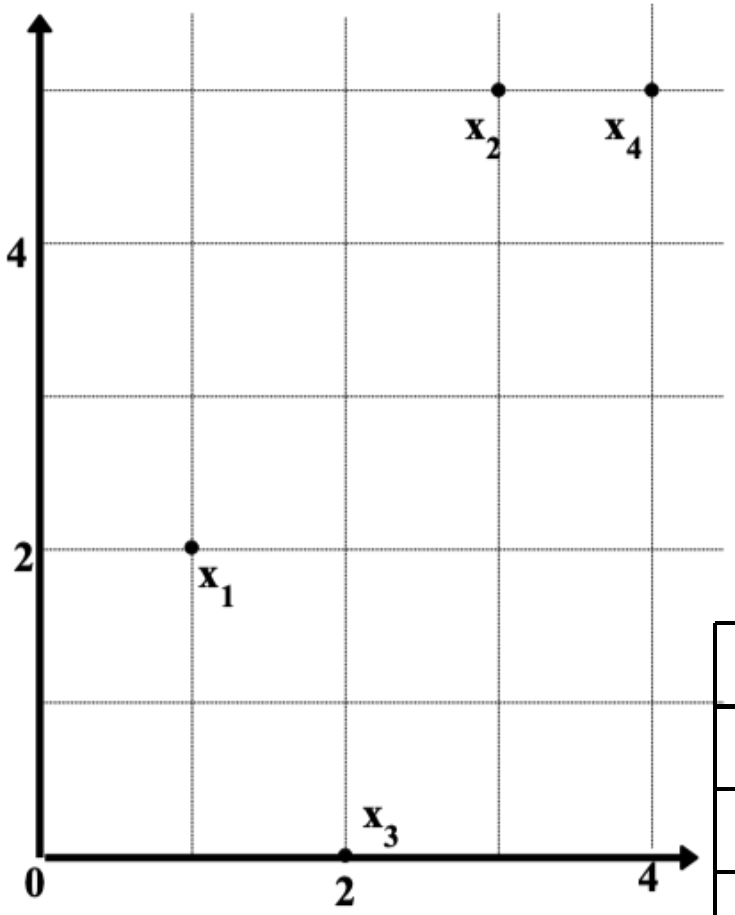
- 性别是对称属性，本题中不考虑其影响
- 其他属性是非对称属性
- 令Y和P值为1, 且N值为0

$$d(jack, mary) = \frac{0+1}{2+0+1} = 0.33$$

$$d(jack, jim) = \frac{1+1}{1+1+1} = 0.67$$

$$d(jim, mary) = \frac{1+2}{1+1+2} = 0.75$$

- **数值属性相异性测度**




**Data Matrix**

| point | attribute1 | attribute2 |
|---|---|---|
| *x1* | 1 | 2 |
| *x2* | 3 | 5 |
| *x3* | 2 | 0 |
| *x4* | 4 | 5 |

**Dissimilarity Matrix**

**(with Euclidean Distance)**

| | *x1* | *x2* | *x3* | *x4* |
|---|---|---|---|---|
| *x1* | 0 | | | |
| *x2* | 3.61 | 0 | | |
| *x3* | 5.1 | 5.1 | 0 | |
| *x4* | 4.24 | 1 | 5.39 | 0 |

- *Minkowski distance*：一种流行的距离测度

$$d(i,j) = \sqrt[h]{|x_{i1}-x_{j1}|^h + |x_{i2}-x_{j2}|^h + \cdots + |x_{ip}-x_{jp}|^h}$$

其中$i=(x_{i1}, x_{i2}, \ldots, x_{ip})$ 和$j=(x_{j1}, x_{j2}, \ldots, x_{jp})$为两个$p$维数据点

特性

- d(i, j)>0 if i≠j, 且d(i,i)=0　　(正定性)
- d(i, j) = d(j, i)　　(对称性)
- d(i, j) < d(i, k) + d(k, j)　　(三角法则)

# 闵可夫斯基距离特殊形式

- $h$ = 1:曼哈顿距离（L1范数）
- Manhattan (city block, $L_1$ norm) distance

$$d(i,j)=|x_{i_1}-x_{j_1}|+|x_{i_2}-x_{j_2}|+...+|x_{i_p}-x_{j_p}|$$

- $h$ = 2: ($L_2$ norm) 欧式距离
- Euclidean distance

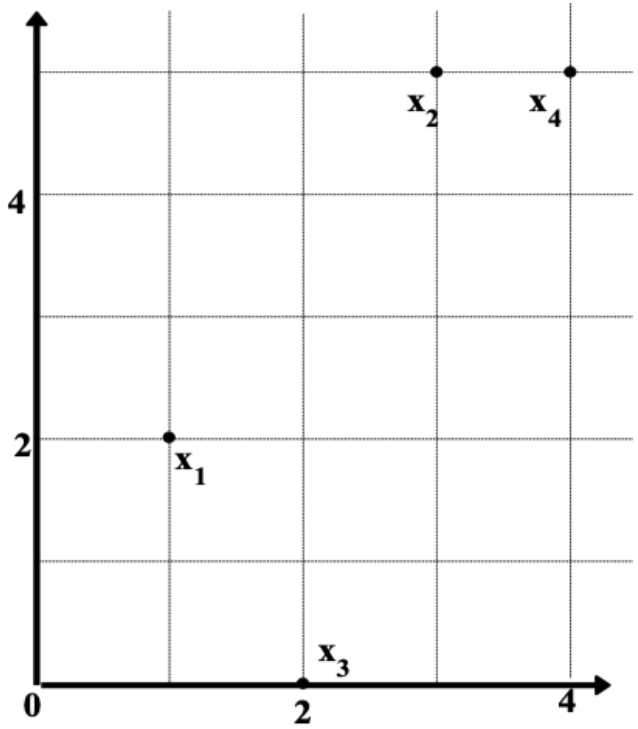
$$d(i,j)=\sqrt{(|x_{i_1}-x_{j_1}|^2+|x_{i_2}-x_{j_2}|^2+...+|x_{i_p}-x_{j_p}|^2)}$$

- $h \to \infty$.上确界距离
- “supremum” ($L_{max}$ norm, $L_\infty$ norm,切比雪夫) distance.

$$d(i,j)=\lim_{h\to\infty}\left(\sum_{f=1}^{p}|x_{if}-x_{jf}|^h\right)^{\frac{1}{h}}=\max_{f}^{p}|x_{if}-x_{jf}|$$

# 计算例子

| point | attribute 1 | attribute 2 |
|---|---|---|
| x1 | 1 | 2 |
| x2 | 3 | 5 |
| x3 | 2 | 0 |
| x4 | 4 | 5 |




曼哈顿距离Manhattan ($L_1$)

| L | x1 | x2 | x3 | x4 |
|---|---|---|---|---|
| x1 | 0 | | | |
| x2 | 5 | 0 | | |
| x3 | 3 | 6 | 0 | |
| x4 | 6 | 1 | 7 | 0 |

欧式距离Euclidean (L2)

| L2 | x1 | x2 | x3 | x4 |
|---|---|---|---|---|
| x1 | 0 | | | |
| x2 | 3.61 | 0 | | |
| x3 | 2.24 | 5.1 | 0 | |
| x4 | 4.24 | 1 | 5.39 | 0 |

上确界距离Supremum

| $L_\infty$ | x1 | x2 | x3 | x4 |
|---|---|---|---|---|
| x1 | 0 | | | |
| x2 | 3 | 0 | | |
| x3 | 2 | 5 | 0 | |
| x4 | 3 | 1 | 5 | 0 |

相异性矩阵Dissimilarity Matrices

# 序数属性的邻近性度量

- 序数属性：有序变量，即一个序变量可以离散的或连续的
  - 用他们的序代替$x_{if}$　　$r_{if} \in \{1,\ldots,M_f\}$
  - 把每一个变量的范围映射到[0,1]，用如下值替代第$f$变量的$i$对象

$$z_{if} = \frac{r_{if} - 1}{M_f - 1} \quad \text{或} \quad z = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

  - 然后对距离进行计算

# 序数属性的邻近性度量-例子

| 对象表示符 | Test-1（标称） | Test-2（序数） | Test-3（数值） |
|---|---|---|---|
| 1 | A | 优秀 | 45 |
| 2 | B | 一般 | 22 |
| 3 | C | 好 | 64 |
| 4 | A | 优秀 | 28 |

假设只考虑test-2来分析对象邻近性度量，

Test2是序数属性，有三个状态，分别是fair、good和 excellent，也就是Mf=3。

1. 把test-2的每个值替换为它的排位，则4个对象将分别被赋值为3、1、2、3。
2. 对排位的标准化，将排位1映射为0，排位2映射为0.5，排位3映射为1.0，属性test-2则变为1，0，0.5，1。
3. 可以用欧几里德距离或者其他距离进行计算，对象1和2最不同。

$$\begin{bmatrix} 0 & & & \\ 1.0 & 0 & & \\ 0.5 & 0.5 & 0 & \\ 0 & 1.0 & 0.5 & 0 \end{bmatrix}$$

- **混合型属性的邻近性度量**

  - 可以用加权法计算合并的影响
    - 如果属性f是二进制（二元）属性，则当$x_{if}=x_{jf}$时 $d^{(f)}_{ij}=0$，否则$d^{(f)}_{ij}=1$
    - 如果属性f是标称属性，则可以化为一系列二元属性；
    - 如果属性f是数值型，则规范化后可以通过距离计算其相异性；
    - 如果属性f是序数属性，则可以通过前面的方法计算其相异性；
    - 最后可以计算其相异度的加权平均值作为混合型属性的邻近性度量。

$$d(i,j)=\frac{\sum_{f=1}^{p}\delta_{ij}^{(f)}d_{ij}^{(f)}}{\sum_{f=1}^{p}\delta_{ij}^{(f)}}$$

# 混合型属性的邻近性度量-例子

| 对象表示符 | Test-1（标称） | Test-2（序数） | Test-3（数值） |
|---|---|---|---|
| 1 | A | 优秀 | 45 |
| 2 | B | 一般 | 22 |
| 3 | C | 好 | 64 |
| 4 | A | 优秀 | 28 |

处理属性test-1（标称）和test-2（序数）的过程与前面所讲类似。 由test-1和test-2得到的相异性矩阵为：

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \qquad \begin{bmatrix} 0 & & & \\ 1.0 & 0 & & \\ 0.5 & 0.5 & 0 & \\ 0 & 1.0 & 0.5 & 0 \end{bmatrix}$$

# 混合型属性的邻近性度量-例子

计算三个属性的相异性矩阵

test-1

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

test-2

$$\begin{bmatrix} 0 & & & \\ 1.0 & 0 & & \\ 0.5 & 0.5 & 0 & \\ 0 & 1.0 & 0.5 & 0 \end{bmatrix}$$

test-3

$$\begin{bmatrix} 0 & & & \\ 0.55 & 0 & & \\ 0.45 & 1.00 & 0 & \\ 0.40 & 0.14 & 0.86 & 0 \end{bmatrix}$$

对每个属性f，如果权重均为1

则：d(3,1)=(1✖1+1✖0.5+1✖0.45)➗3=0.65

混合类型属性相异性矩阵：

$$\begin{bmatrix} 0 & & & \\ 0.85 & 0 & & \\ 0.65 & 0.83 & 0 & \\ 0.13 & 0.71 & 0.79 & 0 \end{bmatrix}$$

# 余弦相似性 Cosine Similarity

- 文档可以用词频来进行量化和表示，如下

| document | team | coach | hockey | baseball | soccer | penalty | score | win | loss | season |
|---|---|---|---|---|---|---|---|---|---|---|
| document1 | 5 | 0 | 3 | 0 | 2 | 0 | 0 | 2 | 0 | 0 |
| document2 | 3 | 0 | 2 | 0 | 1 | 1 | 0 | 1 | 0 | 1 |
| document3 | 0 | 7 | 0 | 2 | 1 | 0 | 0 | 3 | 0 | 0 |
| document4 | 0 | 1 | 0 | 0 | 1 | 2 | 2 | 0 | 3 | 0 |

- 余弦相似性计算公式

$$\cos(d_1, d_2) = (d_1 \bullet d_2) / \|d_1\| \, \|d_2\| ,$$

$$\cos(x, y) = \frac{\sum_{i=1}^{p} x_i y_i}{\sqrt{\sum_{i=1}^{p} x_i^{\,2} \times \sum_{i=1}^{p} y_i^{\,2}}}$$

# 余弦相似性算例: Cosine Similarity

- $\cos(d_1, d_2) = (d_1 \times d_2) / \|d_1\| \|d_2\|$ .
- 算例: 计算文档1和文档2的相似性
- $d_1$ = (5, 0, 3, 0, 2, 0, 0, 2, 0, 0)

  $d_2$ = (3, 0, 2, 0, 1, 1, 0, 1, 0, 1)

  $d_1 \times d_2$ = 5*3+0*0+3*2+0*0+2*1+0*1+0*1+2*1+0*0+0*1 = 25

  $\|d_1\|$ = (5*5+0*0+3*3+0*0+2*2+0*0+0*0+2*2+0*0+0*0)$^{0.5}$=(42)$^{0.5}$=6.481

  $\|d_2\|$ = (3*3+0*0+2*2+0*0+1*1+1*1+0*0+1*1+0*0+1*1)$^{0.5}$=(17)$^{0.5}$=4.123

  $\cos(d_1, d_2)$ = 0.94

| document | team | coach | hockey | baseball | soccer | penalty | score | win | loss | season |
|---|---|---|---|---|---|---|---|---|---|---|
| document1 | 5 | 0 | 3 | 0 | 2 | 0 | 0 | 2 | 0 | 0 |
| document2 | 3 | 0 | 2 | 0 | 1 | 1 | 0 | 1 | 0 | 1 |
| document3 | 0 | 7 | 0 | 2 | 1 | 0 | 0 | 3 | 0 | 0 |
| document4 | 0 | 1 | 0 | 0 | 1 | 2 | 2 | 0 | 3 | 0 |

# 例：一个待估算工作量的项目$P_0$和已完成的项目$P_1$,$P_2$

| Project | Project type | Programming language | Team size | Project size | Effort |
|---|---|---|---|---|---|
| $P_0$ | MIS | C | 9 | 180 | ? |
| $P_1$ | MIS | C | 11 | 200 | 1000 |
| $P_2$ | Real time | C | 10 | 175 | 900 |

| $P_0$ VS. $P_1$ | $P_0$ VS. $P_2$ |
|---|---|
| $d(P_{01},P_{11})=d(MIS,MIS)=0$<br>$d(P_{02},P_{12})=d(C,C)=0$<br>$d(P_{03},P_{13})=d(9,11)=\|(9-11)/(9-11)\|=1$<br>$d(P_{04},P_{14})=d(180,200)$<br>$=\|(180-200)/(200-175)\|=0.8$ | $d(P_{01},P_{21})=d(MIS,Real\ Time)=1$<br>$d(P_{02},P_{22})=d(C,C)=0$<br>$d(P_{03},P_{23})=d(9,10)=\|(9-10)/(9-11)\|=0.5$<br>$d(P_{04},P_{24})=d(180,175)=\|(180-175)/(200-175)\|=0.2$ |
| $distance(P_0,P_1)=1.8/4=0.45$ | $distance(P_0,P_2)=1.7/4=0.425$ |

**由于$P_0$距离$P_1$相对更远，离$P_2$相对更近，因此$P_0$的工作量大概在：**
**1000-（1000-900）×0.45/0.875=900+（1000-900）×0.425/0.875=949**